Python Activity-based learning: 10 Case Studies

NAME: Harshitha

RollNo:26001627

Branch: AIML

Department : CSE

Academic year : 2026 - 2030

Contents:

1. University Admission Decision Engine

2. Traffic Violation Penalty Calculator

3. Laboratory Chemical Inventory Validation

4. Student Timetable Conflict Detection

5. Courier Charge Optimization

6. Campus Event Financial Planning

7. Examination Result and Grace-Mark Validation

8. Hospital Billing and Patient Classification

9. Employee Payroll and Deduction Analysis

10. Multi-Plan Telecommunications Cost Optimization

1.University Admission Decision Engine

1.Statments

Evaluate applicants using subject eligibility, aggregate marks, course-specific thresholds,and a tie-breaking rule. Print eligible applicants in rank order.

2.Objectives

Check required subject eligibility.

Calculate aggregate percentage.

Apply course threshold.

Rank eligible applicants by percentage, then by entrance score.

3.Assumptions

Each applicant has a name, Mathematics mark, Science mark, English mark, and entrance score.

Marks are out of 100; all marks must be at least 0 and at most 100.

Eligibility requires Mathematics ≥ 50 and Science ≥ 50, aggregate ≥ 60%, and entrance score ≥ 40.

Higher aggregate ranks first; ties are broken by higher entrance score. The example uses a fixed maximum

of 20 applicants.

4.Algorithm

1. Start.

2. Input number of applicants (maximum 20).

3. For each applicant, input name, three subject marks, and entrance score.

4. Calculate aggregate percentage and check all eligibility rules.

5. Store eligible records as tuples in a list.

6. Sort the list using simple nested loops: aggregate descending, then entrance score descending.

7. Display ranked eligible applicants or a no-eligible message.

8. Stop.

5.flowchart

Start → Input data → Validate inputs → Process with conditions/loops → Display result →Stop

Decision flow: Start → Input → [Inputs valid?] — No → Display error → Stop; Yes →Calculate/check rules → [Condition met?] → Apply relevant branch → Display output →Stop.

6.pseudocode

START\nINPUT n\nFOR each applicant\n INPUT name, maths, science, english, entrance\n aggregate = (maths + science + english) / 3\n IF maths >= 50 AND science >= 50 AND aggregate >= 60 AND entrance >= 40\n ADD (name, aggregate, entrance) to eligible list\nFOR i from 0 to length-1\n FOR j from i+1 to length-1\n IF j should rank before i, swap tuples\nPRINT ranked eligible list\nSTOP

7.python code

```
n = int(input("Number of applicants (1-20): "))

eligible = []

if n < 1 or n > 20:

print("Invalid applicant count")

else:

for i in range(n):

print("\\nApplicant", i + 1)

name = input("Name: ")

maths = float(input("Mathematics mark: "))
```

```
science = float(input("Science mark: "))
english = float(input("English mark: "))
entrance = float(input("Entrance score: "))
aggregate = (maths + science + english) / 3
if (0 <= maths <= 100 and 0 <= science <= 100 and
0 <= english <= 100 and 0 <= entrance <= 100 and
maths >= 50 and science >= 50 and aggregate >= 60 and
entrance >= 40):
eligible.append((name, aggregate, entrance))
for i in range(len(eligible)):
for j in range(i + 1, len(eligible)):
if (eligible[j][1] > eligible[i][1] or
(eligible[j][1] == eligible[i][1] and eligible[j][2] > eligible[i][2])):
eligible[i], eligible[j] = eligible[j], eligible[i]
print("\\nRank Name Aggregate Entrance")
if len(eligible) == 0:
print("No eligible applicants")
else:
for i in range(len(eligible)):
print(i + 1, eligible[i][0], round(eligible[i][1], 2), eligible[i][2])
```

8.test cases

Input / Test

3 applicants; Asha 80,75,85 entrance 70;

Ravi 65,60,70 entrance 55; Mina 45,90,80

entrance 80

Expected result

Asha ranks before Ravi; Mina is not eligible

because Mathematics < 50.

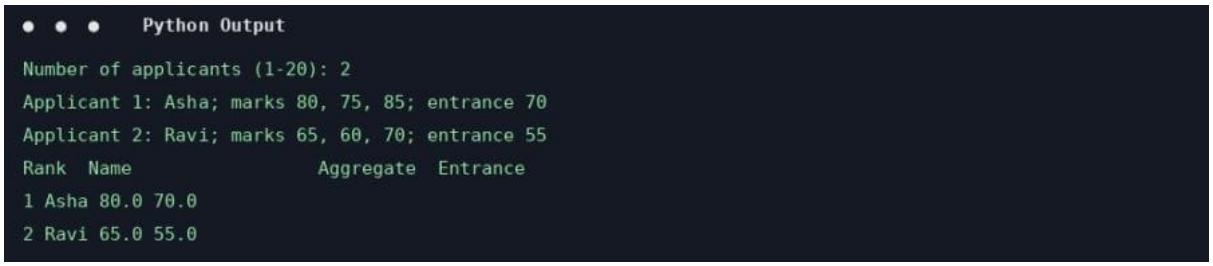
Input / Test

1 applicant with all subject marks 40

Expected result

No eligible applicants.

9.sample output screenshot



Note: This is a sample output illustration. Actual output depends on the values entered.

10.conclusion

The program checks subject-wise and aggregate eligibility, then ranks eligible applicants by

aggregate marks and uses entrance score to break ties.

2. Traffic Violation Penalty Calculator

1. Problem Statement

Calculate a driver's penalty for multiple traffic offences using vehicle category, offence type,

repeat status, and fixed fine rules. Identify disqualification conditions.

2. Objectives

Apply a fine table using conditionals.

Add repeat-offence surcharge.

Calculate total fine.

Flag disqualification when a serious offence is repeated.

3. Assumptions

This is a classroom example, not legal advice or a real jurisdiction's fine schedule.

Vehicle category is car, motorcycle, or commercial.

Base fines: speeding 1000, signal 1500, no-helmet/seatbelt 500.

Repeat offence adds 50% of base fine. Two or more serious offences (signal or speeding) trigger

review/disqualification flag.

4. Algorithm

1. Start and input vehicle category and number of offences.
2. For each offence, input type and whether repeated.
3. Choose base fine from the supplied table.
4. Add 50% if repeat; add to total.
5. Count serious offences and flag if count is at least two.
6. Display total and flag; stop.

5. Flowchart

Start → Input data → Validate inputs → Process with conditions/loops → Display result →Stop

Decision flow: Start → Input → [Inputs valid?] — No → Display error → Stop; Yes →Calculate/check rules → [Condition met?] → Apply relevant branch → Display output →

Stop.

6. Pseudocode

START\nINPUT vehicle, n\nSET total = 0, serious = 0\nREPEAT n times\n INPUT offence,repeat_status\n SELECT base fine from rule table\n IF repeated, fine = base + base * 0.5\ntotal = total + fine\n IF offence is speeding or signal, serious = serious + 1\nIF serious >= 2,print disqualification review\nPRINT total\nSTOP

7. Python Code

```
vehicle = input("Vehicle category (car/motorcycle/commercial): ").lower()

n = int(input("Number of offences: "))

total = 0

serious = 0

if vehicle not in ("car", "motorcycle", "commercial"):

print("Invalid vehicle category")

elif n < 1:

print("Enter at least one offence")

else:

for i in range(n):
```

```
offence = input("Offence (speeding/signal/safety): ").lower()
repeat = input("Repeat offence? (yes/no): ").lower()
if offence == "speeding":
base = 1000
serious = serious + 1
elif offence == "signal":
base = 1500
serious = serious + 1
elif offence == "safety":
base = 500
else:
base = 0
print("Unknown offence; fine is 0")
if repeat == "yes":
fine = base + base * 0.5
else:
fine = base
total = total + fine
print("Offence fine =", fine)
print("Vehicle category:", vehicle)
print("Total penalty =", total)
if serious >= 2:
print("Disqualification review required")
else:
```

print("No automatic disqualification flag")

8. Test Cases

Input / Test

car; speeding repeated; signal not repeated

Expected result

Fine = 1500 + 1500 = 3000; serious

offences = 2; review flag shown.

Input/test

motorcycle; safety not repeated

expected result

Fine = 500; no disqualification flag.

9. Sample Output Screenshot



Note: This is a sample output illustration. Actual output depends on the values entered.

10. Conclusion

The calculator applies the example fine rules consistently, totals multiple offences, and flags

cases needing a disqualification review.

3. Laboratory Chemical Inventory Validation

1. Problem Statement

Validate hypothetical inventory records against storage categories, maximum quantity limits,and compatibility restrictions; report invalid records.

2. Objectives

Check allowed storage categories.

Check maximum quantity

Detect incompatible chemical pairs in one storage unit.

Report valid and invalid records.

3. Assumptions

Hypothetical training rules only; never use this example as real chemical-safety guidance.

Record fields: chemical name, category, quantity in litres, storage unit.

Maximum quantity per record is 20 L.

Acid and base cannot share a storage unit; oxidizer and flammable cannot share a storage unit.

Allowed categories: acid, base, oxidizer, flammable, neutral.

4. Algorithm

1. Input number of records.

2. Input each chemical record and store it as a tuple.

3. For each record, validate category and quantity.

4. Compare each pair sharing a storage unit; flag prohibited category pairs.

5. Print each record and its validation status.

5. Flowchart

Start → Input data → Validate inputs → Process with conditions/loops → Display result →Stop

Decision flow: Start → Input → [Inputs valid?] — No → Display error → Stop; Yes →Calculate/check rules → [Condition met?] → Apply relevant branch → Display output →

Stop.

6. Pseudocode

START\nINPUT n\nFOR each record: INPUT name, category, quantity, unit; store

tuple\nFOR each record\n valid = category allowed AND quantity > 0 AND quantity <= 20\n FOR each other record in same unit\n IF acid/base pair OR oxidizer/flammable pair, valid =false\n PRINT record status\nSTOP

7. Python Code

```
allowed = ("acid", "base", "oxidizer", "flammable", "neutral")

records = []

n = int(input("Number of inventory records: "))

for i in range(n):

print("Record", i + 1)

name = input("Chemical label: ")

category = input("Category (acid/base/oxidizer/flammable/neutral): ").lower()

quantity = float(input("Quantity in litres: "))

unit = input("Storage unit ID: ")

records.append((name, category, quantity, unit))
```

```
for i in range(len(records)):
name, category, quantity, unit = records[i]
valid = True
if category not in allowed or quantity <= 0 or quantity > 20:
valid = False
for j in range(len(records)):
if i != j and records[i][3] == records[j][3]:
pair = (records[i][1], records[j][1])
if pair in (("acid", "base"), ("base", "acid"),
("oxidizer", "flammable"), ("flammable", "oxidizer")):
valid = False
if valid:
print(name, "- VALID")
else:
print(name, "- INVALID: check category, quantity, or compatibility")
```

8. Test Cases

Input / Test

Acid 5 L in S1; base 2 L in S1

Expected result

Both records flagged invalid due to thehypothetical acid/base rule.

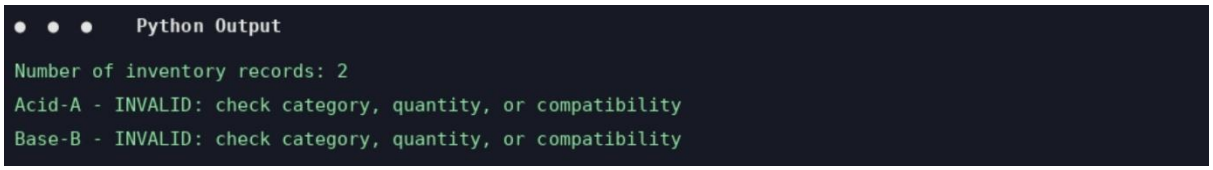
Input/test

Neutral 25 L in S2

Expected result

Record flagged invalid because quantity exceeds 20 L.

9. Sample Output Screenshot



Note: This is a sample output illustration. Actual output depends on the values entered.

10. Conclusion

The program checks quantity and category constraints and compares records sharing a

storage unit to identify rule violations.

4. Student Timetable Conflict Detection

1. Problem Statement

Compare course schedules represented by strings and tuples, identify overlapping time

periods, check room availability, and validate a proposed timetable.

2. Objectives

Represent schedule entries as tuples.

Detect same-day time overlaps.

Detect two classes assigned to the same room at overlapping times.

Report timetable conflicts.

3. Assumptions

Times are integer hour values in a 24-hour day; end time must be greater than start time.

A schedule tuple is (course, day, start, end, room).

Two intervals overlap when start1 < end2 and start2 < end1.

Only sample entries are checked; breaks and travel time are not modelled.

4. Algorithm

1. Input number of classes and their details.
2. Store each class as a tuple.
3. Check each pair of classes.
4. If same day and intervals overlap, flag student conflict.
5. If same room and same day and intervals overlap, flag room conflict.
6. Print results.

5. Flowchart

Start → Input data → Validate inputs → Process with conditions/loops → Display result →Stop

Decision flow: Start → Input → [Inputs valid?] — No → Display error → Stop; Yes →Calculate/check rules → [Condition met?] → Apply relevant branch → Display output →Stop.

6. Pseudocode

START\nINPUT n\nFOR each class, INPUT course, day, start, end, room; store tuple\nSETconflict = false\nFOR each pair i,j where i < j\n IF same day AND start_i < end_j AND start_j< end_i\n PRINT schedule overlap\n IF room_i == room_j, PRINT

room conflict\n SET conflict = true\nIF conflict false, PRINT no conflicts\nSTOP

7. Python Code

```
classes = []
n = int(input("Number of classes: "))
for i in range(n):
print("Class", i + 1)
course = input("Course code: ")
day = input("Day: ").lower()
start = int(input("Start hour (0-23): "))
end = int(input("End hour (1-24): "))
room = input("Room: ")
if start < 0 or end > 24 or start >= end:
print("Invalid time; this entry is still stored for review")
classes.append((course, day, start, end, room))
conflict = False
for i in range(len(classes)):
for j in range(i + 1, len(classes)):
a = classes[i]
b = classes[j]
if a[1] == b[1] and a[2] < b[3] and b[2] < a[3]:
        print("TIME CONFLICT:", a[0], "and", b[0])
conflict = True
if a[4] == b[4]:
```

```
print("ROOM CONFLICT:", a[4])
if conflict == False:
print("No timetable conflicts detected")
```

8.test cases

9. Sample Output Screenshot

Input/test	Expected result
Math Monday 9-10 room A;python Monday 10-12 room A	Time conflict and room conflict
Math Monday 9-10;python Monday 10-11	No overlap because one ends when the other begins



Note: This is a sample output illustration. Actual output depends on the values entered.

10. Conclusion

The program detects overlapping classes and separately reports room conflicts using tuple-based schedule records.

5. Courier Charge Optimization

1. Problem Statement

Calculate delivery prices based on parcel weight, dimensions, distance zone, urgency, and restrictions; compare service options and select the cheapest eligible service.

2. Objectives

Calculate volumetric weight.

Apply weight and zone rates.

Apply urgency surcharge.

Exclude services that cannot accept the parcel.

3. Assumptions

Classroom pricing only; currency units are illustrative.

Volumetric weight = length × width × height / 5000, in kg; chargeable weight is the larger of actual and

volumetric weight.

Zones: local multiplier 1, regional 1.5, national 2.

Standard base rate is 40 per kg; express adds 50%.

Restricted items are not eligible for express service; maximum express chargeable weight is 10 kg.

4. Algorithm

1. Input parcel measurements, zone, urgency, and restriction status.

2. Calculate volumetric and chargeable weight.

3. Calculate standard price.

4. If parcel qualifies for express, calculate express price.

5. Choose the lowest price among eligible services and display comparison.

5. Flowchart

Start → Input data → Validate inputs → Process with conditions/loops → Display result →Stop

Decision flow: Start → Input → [Inputs valid?] — No → Display error → Stop; Yes →Calculate/check rules → [Condition met?] → Apply relevant branch → Display output →Stop.

6. Pseudocode

START\nINPUT weight, length, width, height, zone, urgency, restricted\nvolumetric = length *width * height /5000\nchargeable = max(weight, volumetric)\nzone multiplier = 1, 1.5, or 2\nstandard = chargeable * 40 * multiplier\nIF not restricted AND chargeable <= 10, express= standard * 1.5\nELSE express is unavailable\nCHOOSE cheapest eligible option; PRINT

prices\nSTOP

7. Python Code

```
weight = float(input("Actual weight (kg): "))

length = float(input("Length (cm): "))

width = float(input("Width (cm): "))

height = float(input("Height (cm): "))

zone = input("Zone (local/regional/national): ").lower()

urgency = input("Urgency (standard/express): ").lower()

restricted = input("Restricted item? (yes/no): ").lower()

if zone == "local":

multiplier = 1
```

```
elif zone == "regional":
multiplier = 1.5
elif zone == "national":
multiplier = 2
else:
multiplier = 0
if weight <= 0 or length <= 0 or width <= 0 or height <= 0 or
multiplier == 0:
print("Invalid parcel details")
else:
volumetric = length * width * height / 5000
chargeable = max(weight, volumetric)
standard = chargeable * 40 * multiplier
print("Volumetric weight:", round(volumetric, 2), "kg")
print("Chargeable weight:", round(chargeable, 2), "kg")
print("Standard price:", round(standard, 2))
express_ok = restricted == "no" and chargeable <= 10
if express_ok:
express = standard * 1.5
print("Express price:", round(express, 2))
else:
print("Express service: NOT ELIGIBLE")
if urgency == "express" and express_ok:
```

```
print("Selected service: Express; price =", round(standard * 1.5, 2))

else:

print("Selected service: Standard; price =", round(standard, 2))
```

8. Test Cases

Input / Test

2 kg, 20×20×20 cm, regional, standard, not restricted

Expected result

Volumetric 1.6 kg; chargeable 2 kg; standard price 120.

Input/test

12 kg, local, express, restricted=no

Expected result

Express not eligible because chargeable weight exceeds 10 kg; standard selected.

9.sample output screenshot



Note: This is a sample output illustration. Actual output depends on the values entered.

10. Conclusion

The program compares standard and express options using chargeable weight, zone multiplier, and eligibility restrictions.

6. Campus Event Financial Planning

1. Problem Statement

Calculate event costs for venue, equipment, catering, transport, and certificates; determine per-participant contribution, sponsorship shortfall, and break-even registration count.

2. Objectives

Calculate total fixed and variable costs.

Find the contribution per participant.

Calculate sponsorship shortfall.

Estimate the minimum registrations needed to cover costs.

3. Assumptions

Venue, equipment, and transport are fixed costs; catering and certificates are per participant.

Registration fee is the amount collected from each participant.

Sponsorship is deducted from the total estimated cost.

If contribution per participant is zero or negative, break-even through registrations is not possible.

4. Algorithm

1. Input fixed costs, per-person costs, expected participants, fee, and sponsorship.

2. Calculate total fixed cost and per-person cost.

3. Calculate expected total cost and expected revenue.

4. Calculate sponsorship shortfall if sponsorship is below total cost.

5. Calculate minimum registrations using ceiling division of remaining cost by contribution

per participant.

6. Display results.

5. Flowchart

Start → Input data → Validate inputs → Process with conditions/loops → Display result →Stop

Decision flow: Start → Input → [Inputs valid?] — No → Display error → Stop; Yes →Calculate/check rules → [Condition met?] → Apply relevant branch → Display output →Stop.

6. Pseudocode

START\nINPUT venue, equipment, transport, catering_each, certificate_each, participants,fee, sponsorship\nfixed = venue + equipment + transport\nper_person = catering_each +certificate_each\ntotal_cost = fixed + per_person * participants\nrevenue = fee *participants\nshortfall = max(0, total_cost - sponsorship)\nIF fee > per_person, break_even =ceil(max(0, fixed - sponsorship) / (fee - per_person))\nELSE print break-even not possible\nPRINT totals\nSTOP

7. Python Code

```
import math

venue = float(input("Venue cost: "))

equipment = float(input("Equipment cost: "))

transport = float(input("Transport cost: "))

catering_each = float(input("Catering cost per participant: "))

certificate_each = float(input("Certificate cost per participant: "))

participants = int(input("Expected participants: "))

fee = float(input("Registration fee per participant: "))

sponsorship = float(input("Sponsorship amount: "))
```

fixed = venue + equipment + transport

per_person = catering_each + certificate_each

total_cost = fixed + per_person * participants

revenue = fee * participants

shortfall = max(0, total_cost - sponsorship)

print("Fixed cost =", fixed)

print("Cost per participant =", per_person)

print("Expected total cost =", total_cost)

print("Registration revenue =", revenue)

print("Sponsorship shortfall =", shortfall)

if fee > per_person:

required = max(0, fixed - sponsorship) / (fee - per_person)

print("Minimum registrations to cover costs =",
math.ceil(required))

else:

print("Break-even is not possible with this registration fee")

8. Test Cases

Input / Test

Fixed costs 10000; per-person costs 150;100 people; fee 250; sponsorship 5000

Expected result

Total cost 25000; sponsorship shortfall 20000; minimum registrations to cover fixed cost after sponsorship = 50.

Input/test

Fee 100 and per-person cost 150

Expected result

Break-even is not possible with this registration fee.

9. Sample Output Screenshot



Note: This is a sample output illustration. Actual output depends on the values entered.

10. Conclusion

The planner estimates the event budget, participant revenue, sponsorship gap, and break-even registrations under the stated assumptions.

7. Examination Result and Grace-Mark Validation

1. Problem Statement Process marks across subjects, apply subject-wise pass criteria and a grace-mark policy,and generate a final result without exceeding maximum marks.

2. Objectives

Validate marks against maximum marks.

Apply a limited grace-mark policy.

Check subject pass criteria.

Calculate total, percentage, and classification.

3. Assumptions

Each subject is out of 100 and pass mark is 40.

At most 5 grace marks may be added to one subject that is short by 1–5 marks; only one subject may

receive grace.

Final marks cannot exceed 100.

Classification: Distinction ≥ 75%, First Class ≥ 60%, Second Class ≥ 50%, otherwise Pass if all subjects

passed; otherwise Fail.

4. Algorithm

1. Input subject count and each subject mark.

2. Validate each mark from 0 to 100.

3. Find subjects that are short of 40 by no more than 5 marks.

4. If exactly one eligible subject exists, add only the marks needed to reach 40.

5. Check whether all subjects passed.

6. Calculate total and percentage, then classification.

7. Print subject marks and result.

5. Flowchart

Start → Input data → Validate inputs → Process with conditions/loops → Display result →Stop

Decision flow: Start → Input → [Inputs valid?] — No → Display error → Stop; Yes →Calculate/check rules → [Condition met?] → Apply relevant branch → Display output →Stop.

6. Pseudocode

START\nINPUT n and n marks\nVALIDATE each mark is between 0 and 100\nCOUNT subjects with 35 <= mark < 40\nIF exactly one such subject, raise it to 40\nCHECK every subject >= 40\npercentage = total / (n * 100) * 100\nSET classification by supplied thresholds; fail if any subject failed\nPRINT results\nSTOP

7. Python Code

```
n = int(input("Number of subjects: "))
marks = []
valid = True
for i in range(n):
mark = float(input("Mark for subject " + str(i + 1) + " (0-100): "))
marks.append(mark)
if mark < 0 or mark > 100:
valid = False
if n <= 0 or valid == False:
print("Invalid subject count or marks")
else:
grace_candidates = []
for i in range(n):
if 35 <= marks[i] < 40:
grace_candidates.append(i)
if len(grace_candidates) == 1:
i = grace_candidates[0]
added = 40 - marks[i]
if added <= 5:
marks[i] = 40
print("Grace marks applied:", added, "to subject", i + 1)
total = 0
all_pass = True
```

```
for i in range(n):
total = total + marks[i]
if marks[i] < 40:
all_pass = False
percentage = total / (n * 100) * 100
print("Final marks:", marks)
print("Total =", total, "out of", n * 100)
print("Percentage =", round(percentage, 2))
if all_pass == False:
result = "FAIL"
elif percentage >= 75:
result = "DISTINCTION"
elif percentage >= 60:
result = "FIRST CLASS"
elif percentage >= 50:
result = "SECOND CLASS"
else:
result = "PASS"
print("Final result =", result)
```

8. Test Cases

Input / Test

Marks 80, 70, 38, 90

Expected output

One subject is short by 2; apply 2 grace marks, final marks [80,70,40,90],percentage 70%, First Class.

Input/test

Marks 80, 30, 38

Expected result

Two grace candidates are not allowed; result is FAIL.

9. Sample Output Screenshot



Note: This is a sample output illustration. Actual output depends on the values entered.

10. Conclusion

The result processor applies the stated grace policy once, respects the maximum mark, and reports final percentage and classification.

8. Hospital Billing and Patient Classification

1. Problem Statement

Calculate consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing values, and classify the bill using a supplied payment policy.

2. Objectives

Calculate a patient's gross bill.

Apply concession eligibility.

Validate non-negative charges.

Classify payment status.

3. Assumptions

All charges are illustrative currency amounts.

Senior patients receive 10% off consultation charges only.

If the gross bill is at least 5000, a 5% concession applies to diagnostic charges.

Payment classification: paid amount ≥ final bill means PAID; positive but lower means PARTIAL; zero means

UNPAID.

4. Algorithm

1. Input consultation, diagnostic, and treatment charges, senior status, and paid amount.

2. Validate all amounts are non-negative.

3. Apply consultation and diagnostic concessions when eligible.

4. Calculate final bill and balance.

5. Classify payment status and print statement.

5. Flowchart

Start → Input data → Validate inputs → Process with conditions/loops → Display result →Stop

Decision flow: Start → Input → [Inputs valid?] — No → Display error → Stop; Yes →Calculate/check rules → [Condition met?] → Apply relevant branch → Display output →Stop.

6. Pseudocode

START\nINPUT consultation, diagnostics, treatment, senior, paid\nIF any amount negative,print invalid\nsubtotal = consultation + diagnostics + treatment\nIF senior yes, consultation discount = 10% consultation\nIF subtotal >= 5000,

diagnostics discount = 5%diagnostics\nfinal_bill = subtotal - discounts\nbalance = max(0, final_bill - paid)\nCLASSIFY paid status\nPRINT bill\nSTOP

7. Python Code

```
consult = float(input("Consultation charge: "))

diagnostic = float(input("Diagnostic charge: "))

treatment = float(input("Treatment charge: "))

senior = input("Senior patient? (yes/no): ").lower()

paid = float(input("Amount paid: "))

if min(consult, diagnostic, treatment, paid) < 0:

print("Invalid: charges and payment cannot be negative")

else:

subtotal = consult + diagnostic + treatment

consult_discount = 0

diagnostic_discount = 0

if senior == "yes":

consult_discount = consult * 0.10

if subtotal >= 5000:

diagnostic_discount = diagnostic * 0.05

final_bill = subtotal - consult_discount - diagnostic_discount

balance = max(0, final_bill - paid)

print("Subtotal =", subtotal)

print("Consultation concession =", consult_discount)

print("Diagnostic concession =", diagnostic_discount)
```

```
print("Final bill =", final_bill)
print("Balance due =", balance)
if paid >= final_bill:
print("Payment status: PAID")
elif paid > 0:
print("Payment status: PARTIAL")
else:
print("Payment status: UNPAID")
```

8. Test Cases

Input / Test

Consult 1000, diagnostics 3000, treatment 2000, senior yes, paid 2000

Expected result

Subtotal 6000; discounts 100 and 150; final bill 5750; balance 3750; PARTIAL.

Input/test

All charges zero, paid zero

Expected result

Final bill 0; status PAID under the stated rule.

9. Sample Output Screenshot



Note: This is a sample output illustration. Actual output depends on the values entered.

10. Conclusion

The billing program calculates concessions and outstanding balance and classifies payment

according to the given policy.

9. Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and print a salary statement.

2. Objectives

Calculate total earnings.

Apply progressive deduction slabs.

Find net salary.

Display a formatted payroll statement.

3. Assumptions

All amounts are monthly and illustrative.

Gross = basic + allowances + overtime + incentives.

Deduction: 0% on first 20000; 5% on next 20000; 10% on the amount above 40000.

This is a practice rule, not tax or payroll advice.

4. Algorithm

1. Input salary components.
2. Check that all components are non-negative.
3. Calculate gross salary.
4. Calculate progressive deduction using slabs.

5. Calculate net salary and display statement.

5. Flowchart

Start → Input data → Validate inputs → Process with conditions/loops → Display result →Stop

Decision flow: Start → Input → [Inputs valid?] — No → Display error → Stop; Yes →Calculate/check rules → [Condition met?] → Apply relevant branch → Display output →Stop.

6. Pseudocode

START\nINPUT basic, allowances, overtime, incentives\nIF any value negative, print invalid\n gross = sum of all components\nIF gross <= 20000, deduction = 0\nELSE IF gross<= 40000, deduction = (gross - 20000) * 0.05\nELSE deduction = 20000 * 0.05 + (gross -

40000) * 0.10\nnet = gross - deduction\nPRINT salary statement\nSTOP

7. Python Code

```
basic = float(input("Basic pay: "))

allowances = float(input("Allowances: "))

overtime = float(input("Overtime pay: "))

incentives = float(input("Incentives: "))

if min(basic, allowances, overtime, incentives) < 0:

print("Invalid: salary components cannot be negative")

else:

gross = basic + allowances + overtime + incentives

if gross <= 20000:

deduction = 0
```

```
elif gross <= 40000:
deduction = (gross - 20000) * 0.05
else:
deduction = 20000 * 0.05 + (gross - 40000) * 0.10
net = gross - deduction
print("\\n----- SALARY STATEMENT -----")
print("Basic pay :", basic)
print("Allowances :", allowances)
print("Overtime :", overtime)
print("Incentives :", incentives)
print("Gross salary :", gross)
print("Deduction :", deduction)
print("Net salary :", net)
```

8. Test Cases

Input / Test

Basic 30000, allowances 5000, overtime 2000, incentives 1000

Expected result

Gross 38000; deduction 900; net 37100.

Input/test

All components zero

Expected result

Gross 0; deduction 0; net 0.

9. Sample Output Screenshot



Note: This is a sample output illustration. Actual output depends on the values entered.

10. Conclusion

The payroll program totals earnings, calculates deductions in progressive slabs, and

displays gross and net salary.

10. Multi-Plan Telecommunications Cost Optimization

1. Problem Statement

Evaluate mobile plans against usage, validity, and budget constraints; calculate excess charges and select the most economical eligible plan.

2. Objectives

Compare multiple plan tuples.

Calculate excess data and call charges.

Check validity and budget.

Select the lowest-cost eligible plan.

3. Assumptions

Usage values are GB of data and minutes of calls.

Plan tuples contain (name, price, data allowance, call minutes, validity days, extra-data rate per GB,

extra-call rate per minute).

A plan is eligible only if validity covers required days and total cost does not exceed budget.

Three sample plans are included and can be edited.

4. Algorithm

1. Input usage, required validity, and budget.

2. Create a list of plan tuples.

3. For each plan, calculate excess usage charges and total cost.

4. Check validity and budget.

5. Keep the eligible plan with lowest total cost.

6. Display all plan costs and selected plan.

5. Flowchart

Start → Input data → Validate inputs → Process with conditions/loops → Display result →Stop

Decision flow: Start → Input → [Inputs valid?] — No → Display error → Stop; Yes →Calculate/check rules → [Condition met?] → Apply relevant branch → Display output →Stop.

6. Pseudocode

START\nINPUT data_used, calls_used, days_needed, budget\nSET plans to list of tuples\nSET best_cost to large value\nFOR each plan tuple\n extra_data = max(0, data_used - allowance)\n extra_calls = max(0, calls_used - call_allowance)\n total = price +extra_data * data_rate + extra_calls * call_rate\n IF validity >= days_needed AND total <=budget AND total < best_cost\n save plan as best\nPRINT plan comparisons and best plan\nSTOP

7. Python Code

```
data_used = float(input("Monthly data needed (GB): "))
calls_used = int(input("Call minutes needed: "))
days_needed = int(input("Validity days needed: "))
budget = float(input("Budget: "))
plans = [
("Basic", 199, 2, 300, 28, 50, 0.5),
("Smart", 299, 8, 700, 28, 30, 0.3),
("Max", 399, 15, 1200, 56, 20, 0.2)
]
best_name = ""
best_cost = 999999
for plan in plans:
name, price, data_allow, call_allow, validity, data_rate, call_rate = plan
extra_data = max(0, data_used - data_allow)
extra_calls = max(0, calls_used - call_allow)
total = price + extra_data * data_rate + extra_calls * call_rate
print(name, "total cost =", round(total, 2))
if validity >= days_needed and total <= budget and total < best_cost:
best_name = name
best_cost = total
if best_name == "":
```

print("No plan meets validity and budget requirements")

else:

print("Most economical eligible plan:", best_name)

print("Total estimated cost:", round(best_cost, 2))

8. Test Cases

Input / Test

Data 6 GB, calls 500 min, 28 days, budget 400

Expected result

Basic incurs excess; Smart total 299; Max 399; Smart selected.

Input/test

Data 20 GB, calls 1500 min, 56 days,budget 200

Expected result

No eligible plan meets budget and validity.

9. Sample Output Screenshot



Note: This is a sample output illustration. Actual output depends on the values entered.

10. Conclusion

The plan selector estimates total usage costs and selects the least expensive plan that satisfies validity and budget constraints.